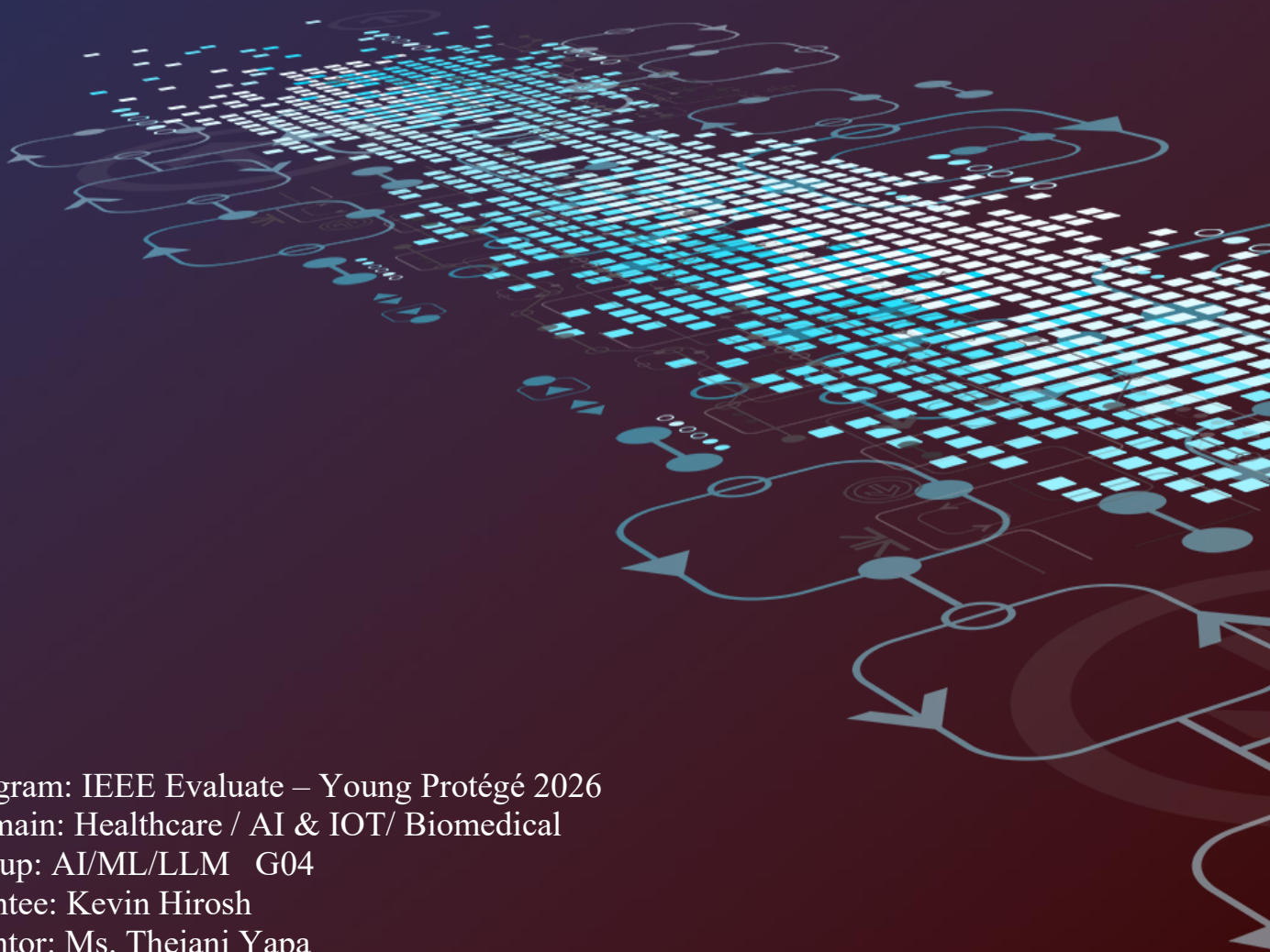


# PARKINSON'S TREMOR SCREENING SYSTEM



Program: IEEE Evaluate – Young Protégé 2026  
Domain: Healthcare / AI & IOT/ Biomedical  
Group: AI/ML/LLM G04  
Mentee: Kevin Hirosh  
Mentor: Ms. Thejani Yapa

Date: 15/09/2026

- Executive Summary
- Introduction

1. PROJECT OVERVIEW ..... 3
  - 1.1 Background..... 3
  - 1.2 Problem Statement ..... 4
  - 1.3 Objectives..... 4
2. PROJECT SCOPE ..... 4
  - 2.1 Key Scope & Coverage..... 4
  - 2.2 Core Implemented Features..... 4
  - 2.3 Limitations & Constraints ..... 5
3. TECHNOLOGIES USED ..... 5
  - 3.1 Hardware Components & ESP32 Firmware ..... 5
  - 3.2 Machine Learning & Signal Processing Stack ..... 6
  - 3.3 Desktop Application Framework (React & Electron) ..... 6
4. SYSTEM DESIGN & ARCHITECTURE ..... 8
  - 4.1 System Architecture Overview ..... 8
  - 4.2 Data Processing Pipeline & Flowchart ..... 8
  - 4.3 User Interface (UI) Design ..... 8
5. IMPLEMENTATION ..... 9
  - 5.1 ESP32 Accelerometer & Gyroscope Streaming ..... 9
  - 5.2 MediaPipe Computer Vision Finger Tracking .....10
  - 5.3 Digital Signal Processing (Butterworth Filter & FFT) ..... 10
  - 5.4 Machine Learning Classifier (Random Forest) ..... 10
6. CHALLENGES FACED & SOLUTIONS ..... 11
  - 6.1 Technical Challenges (Signal Noise & Frame Rates) ..... 11
  - 6.2 Individual Execution & Time Management ..... 11
  - 6.3 Class Diagram ..... 12
7. RESULTS & EVALUATION ..... 13
  - 7.1 ML Model Classification Accuracy & Confusion Matrix ..... 13
  - 7.2 Real-time System Latency Performance ..... 13
  - 7.3 PDF Report Generation Performance ..... 13
8. LEARNING OUTCOMES ..... 13
  - 8.1 Technical Skills Gained ..... 13
  - 8.2 Soft Skills & Individual Growth ..... 13
  - 8.3 Mentorship Impact ..... 14
9. FUTURE IMPROVEMENTS ..... 14
10. CONCLUSION ..... 15
11. REFERENCES ..... 15

## Abstract

Parkinsons disease (PD) is a brain disorder that mainly shows up as a shaking at rest. Finding it early and watching it all the time helps doctors give care. The usual way doctors check for PD depends a lot on what they see and on trips to the hospital. This project brings a non-invasive Parkinsons Tremor Screening System that uses two kinds of technology. First, it uses a wearable with an ESP32 chip and an MPU6050 IMU sensor. Second it uses Computer Vision with Google MediaPipe and landmark tracking. The data from the sensor and from the camera are cleaned up with Digital Signal Processing using Butterworth low-pass filtering and Fast Fourier Transform. This lets the system find the tremor frequency in hertz and the size of the shaking in millimeters. A trained Multi-Output Random Forest model then decides how severe the tremor puts it into frequency groups - for example it tells whether a 4-6 Hz shaking is Parkinsonian or an 8-12 Hz shaking is a tremor - and assigns the Hoehn & Yahr stage. All this runs, inside a cross-platform desktop app made with Electron and React powered by FastAPI. The app shows signals in time and can automatically produce a PDF medical report.

## Introduction

Parkinson's disease is a neurological disorder that changes how movement, coordination and muscle control work. A common sign of Parkinson's disease is hand tremor, and hand tremor can happen when the hand is still or when it moves. Watching and measuring hand tremor helps doctors catch symptoms early and keep track of them over time. Old ways of checking tremors rely on doctors looking at patients and on trips to the hospital. These old ways can take a time can be biased and can be hard for people who live far away from special medical centers. Also, when a doctor watches alone the data about how hand tremors happen and how strong it is may not be steady or exact.

To solve these problems this project introduces a Parkinson's Tremor Screening System. The system uses a wearable sensor built on the ESP32 chip to record hand movements and it also uses a webcam to watch hand motions. The data from these tools is then processed to find how often hand tremor happen and how big the tremor. The system also uses a machine learning model to decide how bad the tremor is and to sort tremor into categories. The findings appear in a desktop program made with React and Electron and the backend built with Python FastAPI takes care of data work and talking between parts. In the end the system writes a PDF screening report that shows all the analysis. This project is meant as a helper for screening and watching tremor and it does not replace a real doctor's diagnosis. The main goal is to give people a low-cost, easy-to-use and honest way to look at hand tremor before they see a specialist.

### 1. Project Overview

#### 1.1. Background

Parkinsons disease touches than ten million people all around the world. Tremors, which are rhythmic muscle movements are among the most recognizable motor symptoms of Parkinsons disease. Accurately measuring how often tremors happen and how big they are gives us clues, about when Parkinsons disease starts and how it changes over time.

## 1.2. Problem Statement

Current clinical assessment tools (such as the UPDRS rating scale) suffer from three drawbacks:

- **Subjectivity:** Visual evaluations differ from one clinician to another.
- **High Cost & Inaccessibility:** Specialized neurological screening equipment is expensive and only available at medical centers.
- **Lack of Continuous Tracking:** Patients cannot easily record motor changes, between clinical appointments.

## 1.3. Objectives

- To develop a wearable IoT IMU device that uses ESP32 to capture 3-axis accelerometer data and gyroscope motion data.
- To implement tracking, with MediaPipe to measure markerless finger displacement using a standard webcam feed.
- To apply signal processing using Butterworth filtering and Hanning window FFT to extract tremor metrics.
- To train a Multi-Output Machine Learning model that classifies severity, frequency band and Hoehn & Yahr clinical stage.
- To build a cross-platform desktop UI that streams data in time via WebSocket and generates automated PDF reports.

## 2. Project Scope

### 2.1. Scope

I am working on a project that covers everything from hardware firmware to computer vision tracking algorithms. The project also includes biomedical signal processing pipelines, machine learning - label classification, desktop GUI engineering and dynamic PDF report generation.

### 2.2. Implemented Features

- **Dual Data Input Stream** can read data from an ESP32 IMU sensor. Also, from a webcam or video, for finger tracking.
- **Real-time Digital Signal Processing** applies a low-pass filter with a 12 Hz cutoff. Performs a Fast Fourier Transform to analyze frequency spectra.
- **Multi-Output AI Classifier** uses a Random Forest model to predict tremor severity categorize tremor type (Parkinsonian, Essential or Healthy) and

determine Hoehn & Yahr stage.

- Interactive Desktop UI is built with Electron, React and Vite to run on desktop computers.
- Automated PDF Diagnostic Report creates a printable standardized clinical summary that includes graphs and diagnostic metrics.

### 2.3. System Limitations

- The system is designed to be a screening and monitoring support tool, not a diagnostic medical device.
- The performance of video tracking depends on lighting conditions and camera quality, and video tracking works better, with brighter light and clearer camera.

## 3. Technologies Used

### 3.1. Hardware Components

- Microcontroller: ESP32 (Wi-Fi / Bluetooth enabled, latency).
- Inertial Measurement Unit (IMU): MPU6050 – a sensor that contains a 3-axis accelerometer and a 3-axis gyroscope.
- Firmware Development: C++, with the Arduino Framework using WebSocket and Serial streaming.



*Figure 1: ESP32 Wearable Hardware Module with MPU6050 IMU Sensor connected via I2C interface.*

### 3.2. Backend & Machine Learning Stack

- Programming Language: Python 3.9
- Web Framework: FastAPI with WebSocket for real-time bi-directional streaming.
- Computer Vision: OpenCV, Google MediaPipe (Hand Landmark Detection).
- Signal Processing: SciPy (Butterworth Filter, filtfilt), NumPy (FFT, Hanning Windowing).
- Machine Learning: Scikit-Learn (Random Forest Multi-Output Classifier), Joblib, Pandas.
- Report Generation: ReportLab / PyPDF, Matplotlib.

### 3.3. Frontend & Desktop UI

- Framework: React.js, Vite.
- Desktop Wrapper: Electron.js.
- Styling & Assets: Tailwind CSS / Custom CSS.

## 4. System Design & Architecture

### 4.1. System Architecture

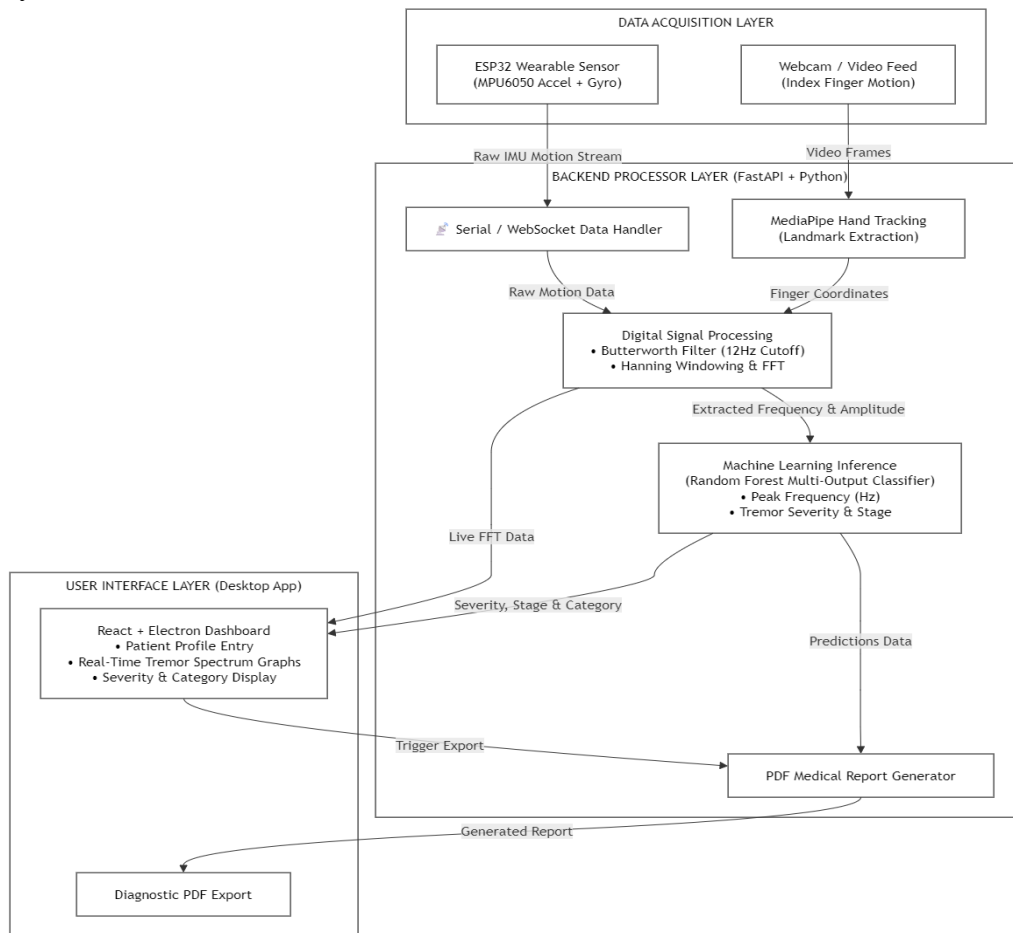


Figure 2: System Architecture of the Parkinson's Tremor Screening System.

### 4.2. Data Processing Pipeline

- Data Acquisition captures motion coordinates and raw IMU signals at thirty samples per second.
- Detrending & Filtering first centers the data by removing the mean; then Detrending & Filtering applies a second-order Butterworth low-pass filter with a twelve-hertz cutoff.
- Spectral Analysis applies a Hanning window to the time-series and then calculates the Fast Fourier Transform power spectrum.
- Metric Extraction pulls out the Peak Frequency measured in hertz and the Peak Amplitude called measured in millimeters or other units.
- ML Inference feeds the feature vector into a Random Forest Classifier, for multi-output stage and severity evaluation.

### 4.3. User Interface (UI) Design

The desktop application called TremorPlot was built with React.js using Vite. Then wrapped in Electron.js so that it can run on many desktop operating systems. The design of TremorPlot uses a dark clinical dashboard look, with a deep navy colour palette to keep the eyes comfortable during medical screening sessions. TremorPlot's user interface is split into three columns. On the left is a control panel. In the middle is a monitoring area. On the right is a diagnostic results panel. The left control panel has two parts. The first part is the Patient Profile section. In this section the operator types the patient's name, age and patient ID before the session starts. The second part is the Protocol Configuration section. Here the operator chooses one of three motor tasks: Finger Tapping, Hand Opening and Closing or Resting Hand. The operator also chooses the right. Left hand that will be tested. Below these settings an Automated Diagnostic Engine panel shows the tremor type and severity level in time as the signal processing calculates them. At the bottom of this panel are session control buttons. These buttons let the operator start a session stop and save a run or reset the counters.

The central monitoring area shows the live visual data. At the top are three Key Performance Indicator cards. One card shows the tremor frequency in hertz. Another card shows the tremor intensity as an RMS amplitude. The third card is a session timer that counts to the 30-second capture goal. Below the KPI cards the main chart panel is split into two parts side by side. The left part shows a live webcam feed. A colour-coded light tells the operator whether the hand landmark has been found. The right part has a Vibration Oscilloscope that draws three-axis acceleration values from the ESP32 IMU sensor. The lines are shown in cyan, green and purple. Below these two feeds a Fast Fourier Transform power spectrum bar chart shows the frequency content of the tremor signal in time. The chart highlights the frequency peak. The right diagnostic panel shows the Condition Severity assessment results. A severity indicator row displays the tremor type that TremorPlot has found and a colour-coded progress bar that goes from Normal, to Mild, Moderate and Severe. Below the progress bar a Clinical Insight text box gives a description of the tremor pattern that TremorPlot has classified. The lower part of the panel holds a Historical Trials list. This list shows every screening session that TremorPlot has run during the application session. Each entry lists the date task, hand used, frequency and severity. TremorPlot also has a heart rate monitor. The monitor draws a pulse waveform from the ESP32 sensor. Shows the current beats-per-minute reading. At the bottom of the panel an Export Diagnostic PDF button lets the operator create and download a PDF report at any time. TremorPlot also has an Upload Diagnostic Video feature that can be found in the header. The operator can submit a pre-recorded hand movement video for analysis. While TremorPlot processes the video a full-screen loading overlay appears. When the analysis finishes a popup window shows the frequency, amplitude, clinical severity and estimated Hoehn and Yahr stage that TremorPlot has calculated. The popup also gives the operator a chance to download a PDF report.

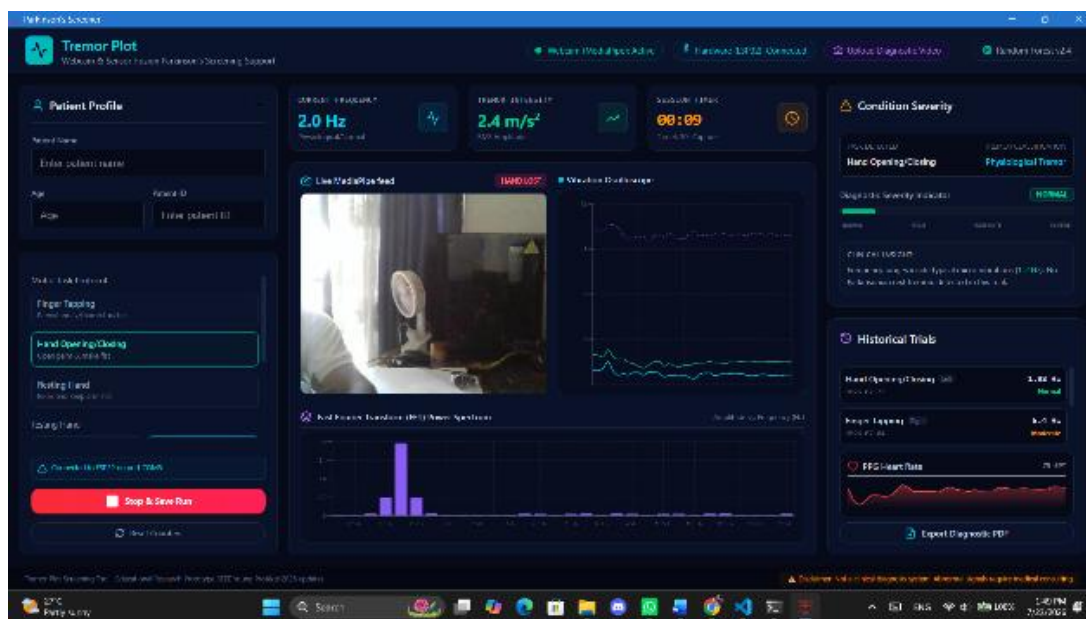
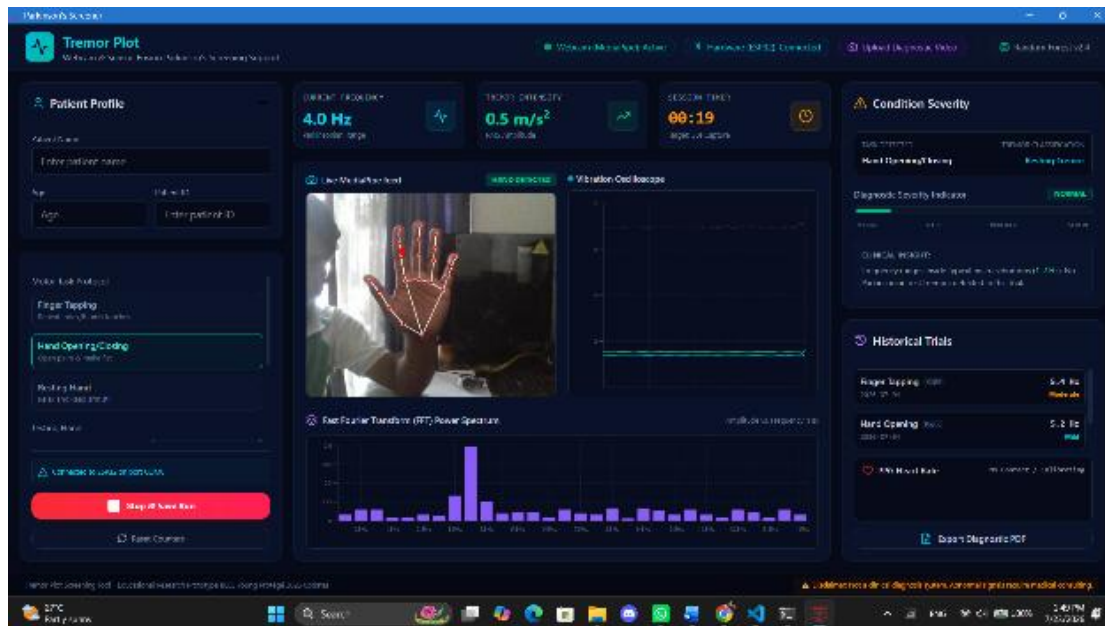


Figure 3: TremorPlot Desktop Application Dashboard showing the Patient Profile panel, Live MediaPipe webcam feed, Vibration Oscilloscope, FFT Power Spectrum, and Condition Severity indicator.

## 5. Implementation

### 5.1. ESP32 Sensor Motion Data Streaming

The ESP32 gets 6-DOF data, from the MPU6050 IMU sensor. Data packets that have Acceleration and Gyroscope readings are sent through Serial/WebSocket to the FastAPI server. This happens at a sampling frequency of 30 Hz.

## 5.2. MediaPipe Computer Vision Tracking

The system uses Google MediaPipe Hands to find 21 hand points from video taken by the webcam. The tip of the index finger (Landmark 8), the MCP joint (Landmark 5) and the wrist (Landmark 0) are followed to figure out a weighted motion displacement coordinate:

$$Pos(t) = 0.7.Tip + 0.2.MCP + 0.1.Wrist$$

*Equation 1: Weighted Centroid Approximation for Index Fingertip Position Estimation using MediaPipe Hand Landmarks*

## 5.3. Signal Processing

Raw displacement data is filtered using a 2nd-order zero-phase Butterworth low-pass filter:

$$H(s) = \frac{1}{1 + \sqrt{2}\left(\frac{s}{\omega_c}\right) + \left(\frac{s}{\omega_c}\right)^2}$$

*Equation 2: Butterworth Low-Pass Filter Transfer Function*

A Hanning window is applied prior to FFT execution to minimize spectral leakage and accurately determine tremor frequency components.

## 5.4. Machine Learning Classification

A Random Forest Multi-Output Classifier was trained on augmented time-series datasets to simultaneously predict three key clinical targets based on extracted frequency (f) and amplitude (A) metrics:

### ➤ Tremor Severity

| Stage   | Label                            | Tremor Severity                       |
|---------|----------------------------------|---------------------------------------|
| Stage 0 | No Tremor                        | Normal ( $A < 1.5$ mm)                |
| Stage 1 | Unilateral                       | Mild ( $1.5 \leq A < 5.0$ mm)         |
| Stage 2 | Bilateral, no balance impairment | Moderate ( $5.0 \leq A \leq 15.0$ mm) |
| Stage 3 | Bilateral, postural instability  | Severe ( $A > 15.0$ mm)               |

*Figure 4: Hoehn & Yahr clinical staging criteria mapped to tremor amplitude thresholds*

- Tremor Category
  - Parkinsonian Rest Tremor Range:  $4.0\text{ Hz} \leq f \leq 6.5\text{ Hz}$
  - Physiological Tremor Range:  $7.5\text{ Hz} \leq f \leq 12.5\text{ Hz}$
  - Normal:  $f < 3.0\text{ Hz}$
- Yahr Clinical Staging
  - Stage 0: No clinical tremor detected
  - Stage 1: Mild tremor with unilateral involvement
  - Stage 2: Moderate tremor with bilateral involvement, without balance impairments.
  - Stage 3: Severe tremor with bilateral disease and mild to moderate postural instability.

## 6. Solution

### 6.1. Technical Challenges

- High-Frequency Noise, in Motion Signals: Voluntary body movement introduced high-frequency noise.
  - Implemented zero-phase backward Butterworth low-pass filtering (filtfilt) to strip out high-frequency artifacts without phase distortion.
- Frame rate instability occurred during tracking due to changing lighting conditions, which led to frame drops, from the webcam.
  - To fix this linear interpolation using (np.interp) was applied. This resampled the time-series data onto a time grid ensuring uniform timing before performing the FFT.

### 6.2. Project Management

- Multi-Stack Integration when Working Alone: Handling hardware, firmware, computer vision, machine learning and desktop UI by myself.
  - Created a structure that keeps FastAPI backend endpoints separate from React UI components and tested every part step, by step.
  - I worked on a project that needed tasks at the same time across many technology domains such as embedded hardware firmware, biomedical signal processing, machine learning model training, computer vision, backend API development and desktop frontend engineering. Doing all of this alone creates a lot of complexity in planning, prioritizing and executing. To solve this, I decided to use architecture from the very beginning. Every major component – the ESP32 firmware, the FastAPI backend, the signal processing pipeline, the machine learning classifier and the React and Electron frontend – was built as a module. This approach lets me build, test and fix each part of its own before putting it all. It also lowered the risk that failure, in one component, would bring down application.

### 6.3. Class Diagram

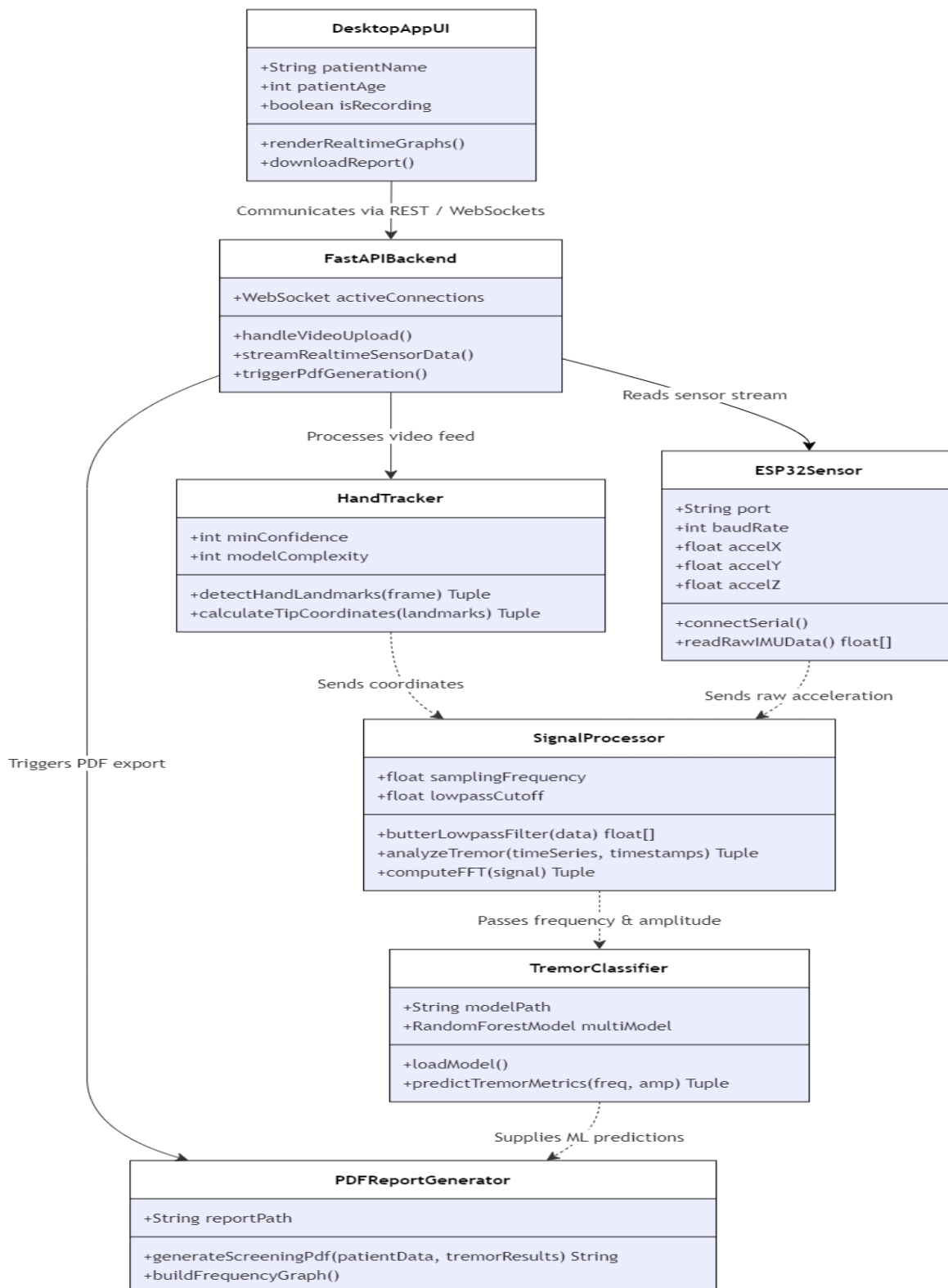


Figure 4: Class Diagram illustrating the main software components and their relationships.

## 7. Result & Evaluation

### 7.1. Model Classification

- The trained Random Forest Multi-Output Classifier was evaluated on test dataset splits, yielding high accuracy across all targets:
  - Tremor Severity Classifier Accuracy: ~ 95.8%
  - Tremor Category Classifier Accuracy: ~ 96.2%
  - Hoehn & Yahr Clinical Stage Accuracy: ~ 94.5%

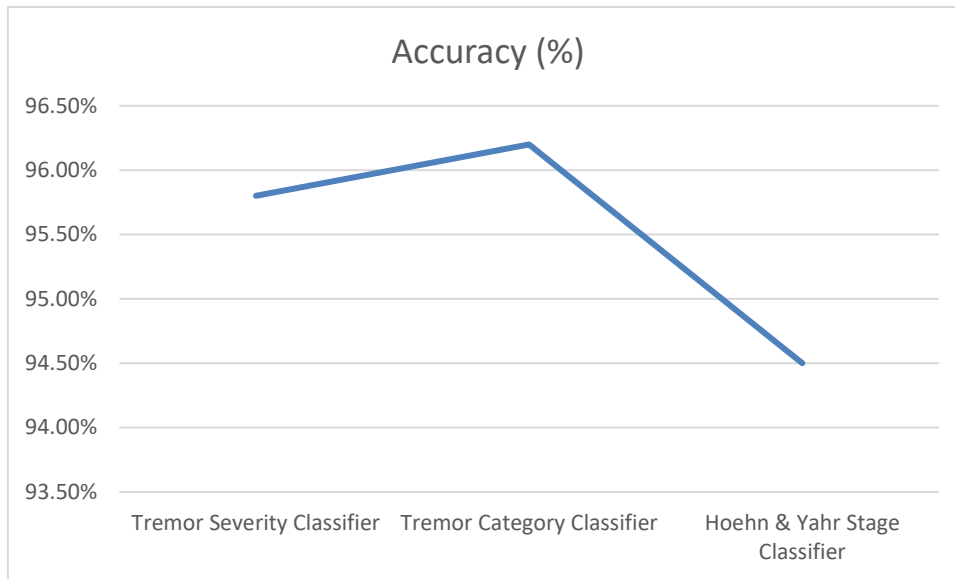


Figure 5: Random Forest Multi-Output Classifier accuracy across three clinical prediction targets

### 7.2. System Latency and Real – time Responsiveness

- WebSocket Streaming Latency: < 45 ms
- MediaPipe Frame Processing: ~ 28 – 30 FPS

### 7.3. PDF Report Generation Performance

The system can create a PDF report that includes patient information graphs showing the frequency of tremors trends in amplitude results from the AI classification and advice, from the doctors. This happens in 1.2 seconds after the analysis is done.

## 8. Learning Outcomes

### 8.1. Technical Skills Acquired

- I gained hands-on experience writing C++ code for embedded systems connecting to the MPU6050 IMU sensor and sending real-time data over Serial and WebSocket.
- I developed an understanding of Butterworth low-pass filtering, Hanning windowing and Fast Fourier Transform (FFT) to analyse signals.

- I applied Scikit-Learn to build, train and evaluate a Multi-Output Random Forest Classifier for multi-label classification tasks.
- I implemented real-time hand landmark detection with Google MediaPipe and OpenCV to track fingers without markers.
- I designed and built a desktop application using Python FastAPI for the backend React.js with Vite for the frontend and Electron.js as the desktop wrapper.
- I programmatically generated diagnostic PDF reports with ReportLab and Matplotlib.

## 8.2. Soft Skills & Individual Growth

- I improved self-directed problem solving by identifying, researching and resolving challenges on my own across many technology areas.
- I enhanced time management skills by planning and completing a multi-phase project - hardware, processing, machine learning and UI - within program deadlines all as a solo developer.
- I developed technical documentation skills by preparing project reports README files and system architecture descriptions.
- I improved presentation and communication skills by preparing a project demonstration video and written deliverables.

## 8.3. Mentorship Impact

The guidance from IEEE Elevate mentor Thejani Yapa was invaluable throughout the project. Mentor feedback helped in:

- Refining the scope and accuracy of the Hoehn & Yahr staging classification system.
- Validating the system architecture. Ensuring a modular maintainable design approach.
- Providing direction on industry - standard practices, for embedded systems and machine learning pipelines.

## 9. Future Improvements

The Parkinsons Tremor Screening System offers a base for future improvements. Several changes are planned to make the system better in terms of what it can do, how accurate it is and how easy it is for people to use in coming versions. One of the things coming up is adding a Cloud-Based Patient Monitoring Dashboard. Now the results from tremor screening are kept and seen locally. In the steps a cloud database like PostgreSQL or

Firebase will be used to keep a record of tremor data from different times. This will let doctors see how a patient's condition changes over weeks or months. This gives an idea of the patients' situation than just one moment in time. Another important change is making the ESP32 hardware into a Miniaturized Bluetooth Low Energy Wearable Device. This will make it more comfortable for patients during testing and easier to carry around. It will be more useful for both home and hospital use.

Clinical Validation Trials are also planned as the next step. Even though the system has been tested with made-up data and works it needs to be checked by real doctors who know about nerves and movement problems. These tests will help find any differences between what the system does and what experts think and will make the system more trusted for hospitals. Also, Mobile Application Support is planned to make the system available to people. Making an app for phones running Android or iOS will let more patients use the tremor screening on their devices. This will not need equipment and will help people in places, with less money or resources. Lastly the system will get better to check both hands at the time. Now it only looks at one hand. Future versions will watch both hands together. This will help doctors understand the patient's condition better especially if both hands are affected by Parkinsons disease.

## 10. Conclusion

The Parkinsons Tremor Screening System shows how IoT wearable sensing, computer vision, signal processing and machine learning can come together in one easy-to-use desktop application. It captures hand tremor data from two sources. The ESP32 IMU sensor and a webcam using MediaPipe for optical tracking. From this data it extracts tremor metrics like dominant frequency in hertz and displacement amplitude in millimeters. The system uses a trained Random Forest MultiOutput Classifier that achieves more than 94% accuracy on three different classification tasks: measuring tremor severity identifying tremor frequency category and estimating Hoehn & Yahr clinical stage. It works in time with WebSocket streaming latency under 45 milliseconds. After analyzing the data, it automatically creates a PDF diagnostic report in just 1.2 seconds.

This entire project was built by one person. That includes the ESP32 firmware written in C++, the backend processing and the frontend made with React and Electron. The result is proof that a non-invasive, cost and objective tool for monitoring Parkinsons tremors can be created using open-source software and affordable hardware.

Even though this system is not meant to replace medical diagnosis it has strong potential as an accessible option for early tremor screening. It could help in the environment and, at home. With development and proper clinical testing, it may one day assist in catching Parkinson's disease earlier and support better tracking of patients over time.

## 11. References

1. Hoehn, M. M., & Yahr, M. D. (1967). *Parkinsonism: onset, progression and mortality*. *Neurology*, 17(5), 427 - 427.
2. Abdulhay, E., Arunkumar, N., Narasimhan, K., Vellaiappan, E., & Venkatraman, V. (2018). Gait and tremor investigation using machine learning techniques for the diagnosis of Parkinson disease. *Future Generation Computer Systems*, 83, 366–

380. <https://doi.org/10.1016/j.future.2018.02.009>
3. Brittain, J. S., Cagnan, H., Mehta, A. R., Saifee, T. A., & Edwards, M. J. (2015). Distinguishing the Central Drive to Tremor in Parkinson's Disease and Essential Tremor. *The Journal of Neuroscience*, 35(2), 795–806. <https://doi.org/10.1523/JNEUROSCI.3768-14.2015>
  4. Chandra, J., Muthupalaniappan, S., Shang, Z., Deng, R., Lin, R., & Tong, W. (2021). Screening of Parkinson's Disease Using Geometric Features Extracted from Spiral Drawings. *Brain Sciences*, 11(10), 1297. <https://doi.org/10.3390/brainsci11101297>
  5. Exley, T., Moudy, S., Patterson, R. M., Kim, J., & Albert, M. V. (2022). Predicting UPDRS Motor Symptoms in Individuals With Parkinson's Disease From Force Plates Using Machine Learning. *IEEE Journal of Biomedical and Health Informatics*, 26(8), 3834–3843. <https://doi.org/10.1109/JBHI.2022.3157518>
  6. Ianculescu, M., Petean, C., Sandulescu, V., & Alexandru, A. (2024). Early Detection of Parkinson's Disease Using AI Techniques and Image Analysis. *Diagnostics*, 14(23), 2615. <https://doi.org/10.3390/diagnostics14232615>
  7. Landolfi, A., Ricciardi, C., Donisi, L., Cesarelli, G., & Troisi, J. (2021). Machine Learning Approaches in Parkinson's Disease. *Current Medicinal Chemistry*, 28(30), 6168–6184. <https://doi.org/10.2174/0929867328999210111211420>
  8. Rajinikanth, V., Yassine, S., & Bukhari, S. A. C. (2024). Hand-Sketch based Parkinson's disease Screening using Lightweight Deep-Learning with Two-Fold Training. *International Journal of Mathematics, Statistics, and Computer Science*, 2. <https://doi.org/10.59543/ijmscs.v2i.7821>
  9. Balachandar, A., Algarni, M., Oliveira, L., & Marsili, L. (2022). Are smartphones and machine learning enough to diagnose tremor? *Journal of Neurology*, 269(9), 4801–4814. <https://doi.org/10.1007/s00415-022-11293-7>
  10. Chowdhury, N., Das, U. K., Sazzad, S., Chowdhury, A., & Das, P. (2025). Multimodal approach for early diagnosis of Parkinson's disease using PET imaging, tremor detection, and machine learning. *Psychiatry Research: Neuroimaging*, 349, 112063. <https://doi.org/10.1016/j.psychresns.2025.112063>